Введение

Что такое микросервисы?

Микросервисная архитектура - подход к разработке программных систем как набора небольших, независимо развёртываемых сервисов, каждый из которых:

- Работает в собственном процессе
- Владеет собственными данными
- Общается с другими сервисами по сети (HTTP/REST, gRPC, асинхронные сообщения)
- Организован вокруг бизнес-возможности

Почему именно эти четыре характеристики?

- Собственный процесс - не просто модуль в одном приложении. Это отдельная единица развёртывания. В Google, например, во многом до сих пор каждый сервис - отдельный бинарник, работающий в контейнере Borg (предшественник Kubernetes)
- Владение данными - самый важный и самый часто нарушаемый принцип. Если два сервиса владеют одной базой данных - это не микросервисы, а «распределённый монолит». В Amazon каждый сервис владеет своей таблицей в DynamoDB; в Google - своим Spanner namespace или Bigtable

Почему именно эти четыре характеристики?

- Сетевое взаимодействие - фундаментальное отличие от модульного монолита. Вызов метода в памяти заменяется сетевым вызовом. Это не просто разница в скорости — это принципиально другая модель отказов (network partition, timeout, retry)
- Бизнес-возможность - сервис не «сервис авторизации» и не «сервис базы данных». Это «сервис управления заказами», «сервис каталога товаров». Границы сервисов совпадают с границами бизнес-доменов (Domain-Driven Design)

Что такое микросервисы?

Микросервисы - небольшие, автономные, совместно работающие сервисы. Небольшие и нацеленные на то, чтобы хорошо справляться только с одной работой

Сэм Ньюман, автор «Building microservices» и других книг о микросервисах

Что такое микросервисы?

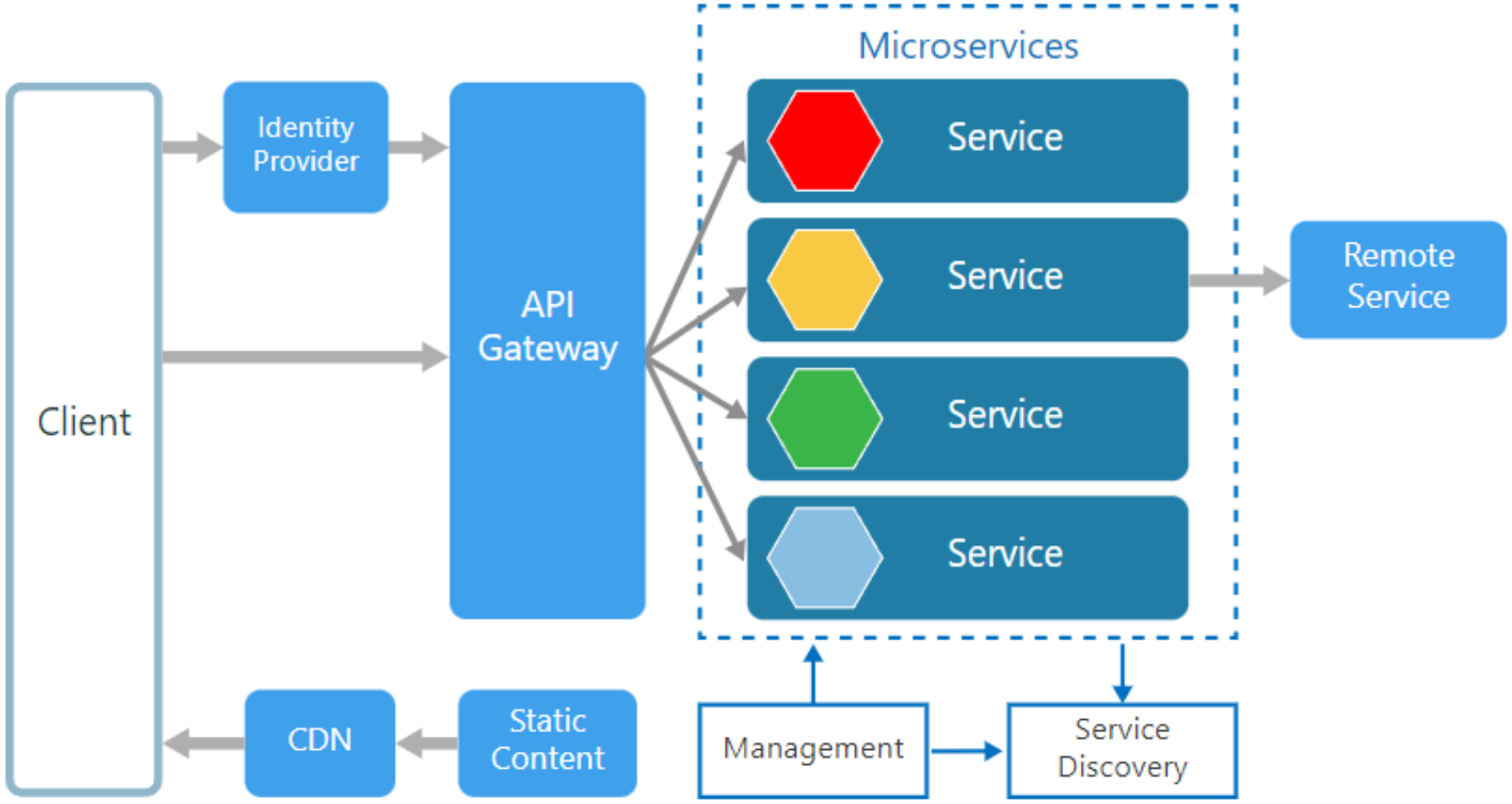
Архитектурный стиль, при котором система строится как набор небольших сервисов, каждый из которых работает в своём процессе, взаимодействует через облегчённые механизмы и развёртывается независимо

Мартин Фаулер, Джеймс Льюис

Эволюция архитектур

Характеристика	Монолит	SOA	Микросервисы
Размер сервиса	Одно приложение	Крупные сервисы	Небольшие сервисы
Данные	Одна БД	Общие БД	БД на каждый сервис
Связь	Вызовы в памяти	ESB, SOAP	REST, gRPC, брокеры
Развёртывание	Целиком	Относительно независимо	Полностью независимо
Команды	Одна большая	Несколько крупных	Много малых
Управление	Централизованное	Централизованное (ESB)	Децентрализованное

Типичная архитектура микросервисного приложения



Закон Конвея

Организации, проектирующие системы, ограничены дизайном, который копирует коммуникационную структуру самой организации

Мелвин Конвей, 1967

- Структура команд определяет структуру сервисов
- Одна команда → один или несколько связанных сервисов
- Команда владеет сервисом от разработки до продакшена («you build it, you run it»)
- Если два сервиса требуют постоянной координации между командами - возможно, граница проведена неверно

Почему это важно?

- Микросервисы - это не технологическое, а организационное решение
- Это реализуется через модель product areas: каждая предметная область (ads, search, cloud ....) имеет свои команды, каждая команда владеет своими сервисами
- Решение о том, как нарезать сервисы, принимается не на основе технических слоёв (UI, бизнес-логика, данные), а на основе бизнес-доменов

Почему это важно?

- «You build it, you run it» - фраза Вернера Вогельса (CTO Amazon), ставшая мантрой микросервисного мира.
- Команда, написавшая сервис, несёт ответственность за его работу в продакшене, поэтому она дежурит, реагирует на инциденты, оптимизирует производительность.
- Это радикально меняет мотивацию, если тебе придётся в 3 часа ночи чинить сервис, который ты написал, будешь думать а как бы не обосра.... *ты напишешь его надёжнее*

Обратный манёвр Конвея

Вместо того чтобы подстраивать архитектуру под оргструктуру, компании перестраивают оргструктуру под целевую архитектуру

Вопрос

Представим три команды - frontend, backend, database. Какую архитектуру они создадут?

Вопрос

Теперь представьте команды - заказы, платежи, каталог. А тут архитектура?

Принципы микросервисной архитектуры

- Единая ответственность (Single Responsibility)
  - Каждый сервис делает одно дело и делает его хорошо
- Автономность (Autonomy)
  - Сервис можно разработать, протестировать, развернуть и масштабировать независимо
- Владение данными (Data Ownership)
  - Каждый сервис единственный владелец своих данных, другие обращаются только через API
- Устойчивость к отказам (Design for Failure)
  - Сетевые вызовы ненадёжны, сервис должен корректно работать при сбоях зависимостей

Принципы микросервисной архитектуры

- Наблюдаемость (Observability)
  - Если вы не можете это наблюдать, вы не можете этим управлять
- Автоматизация (Automation)
  - CI/CD, инфраструктура как код, автоматическое масштабирование уже необходимость
- Децентрализованное управление (Decentralized Governance)
  - Команды сами выбирают технологии, подходящие для задачи

Технологическая разнородность

- Каждый сервис может использовать стек, оптимальный для его задачи:

Сервис	Язык	БД	Обоснование
Каталог товаров	Java/Spring	Elasticsearch	Полнотекстовый поиск
Корзина	Go	Redis	Высокая скорость, простая модель данных
Аналитика	Python	ClickHouse	ML-экосистема, колоночное хранение
Платежи	Java	PostgreSQL	Надёжность, ACID-транзакции

Разнородность имеет цену, каждый стек нужно поддерживать

Компромиссы и разнородность

- В теории каждый сервис может быть на своём языке. На практике это приводит к проблемам - сложно найти специалистов, сложно переводить людей между командами, сложно поддерживать библиотеки для каждого языка
- Поэтому обычно выбирают 2-3 основных языка, не больше
- Самый эффективный аспект разнородности – хранилища данных.
  - Выбор базы данных под задачу даёт ощутимый выигрыш:
    - Поиск по текстовым данным в Elasticsearch в 100x быстрее, чем LIKE-запросы в PostgreSQL
    - Redis обслуживает 100K+ операций в секунду для кэширования
    - ClickHouse обрабатывает аналитические запросы по миллиардам строк за секунды

Вопрос

Когда технологическая разнородность приносит больше вреда, чем пользы?

Независимое развёртывание

- Монолит
  - Изменение в одном модуле → пересборка и развёртывание всего приложения → риск регрессии во всём приложении
- Микросервисы
  - Изменение в одном сервисе → развёртывание только этого сервиса → в целом изолированный риск
  - Критическое условие - обратная совместимость API.
    - Если сервис A зависит от сервиса B, и сервис B меняет формат ответа - сервис A сломается. T
    - Поэтому версионируем API (v1, v2), игнорируем неизвестные поля, проверяем на пригодность и т.д.

Гранулярное масштабирование

- Монолит
  - Узкое место в одном модуле → масштабируем всё приложение целиком
- Микросервисы
  - Узкое место в одном сервисе → масштабируем только его
  - Достигаем экономии на инфраструктуре до 30-60% по сравнению с равномерным масштабированием монолита (если его вообще возможно масштабировать)

- Во многом в системах оркестрации (Kubernetes HPA (Horizontal Pod Autoscaler) возможно масштабирование по CPU, памяти или кастомным метрикам автоматически

Реальный кейс на посмеяться

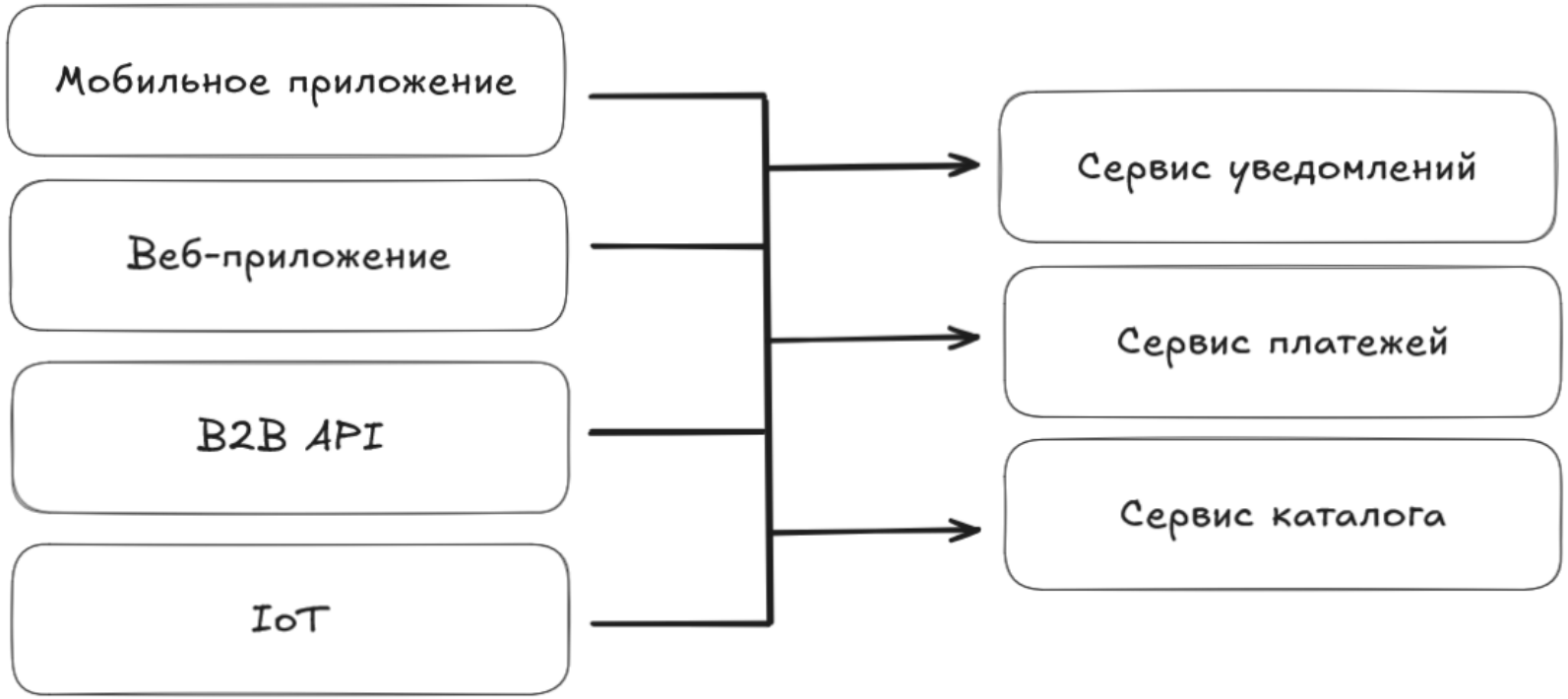
- В монолите появился ИИ для обработки изображений и требовал GPU
  - Любое горизонтальное масштабирование выходит уже в копеечку
- В микросервисной архитектуре только сервис обработки изображений будет работать на GPU, остальные - на дешёвых стандартных машинах

Устойчивость к отказам

- Монолит
  - Сбой в одном модуле → падает всё приложение
- Микросервисы
  - Сбой в одном сервисе → деградация, но система продолжает работать
  - Паттерны устойчивости:
    - Circuit Breaker - если зависимый сервис не отвечает, прекращаем попытки обращения на заданное время (как автоматический выключатель в электрике)
    - Retry + Exponential Backoff - повторяем запрос с возрастающей задержкой (100мс, 200мс, 400мс...)
    - Graceful Degradation - показываем кэшированные данные или упрощённый ответ вместо ошибки
    - Timeout - никогда не ждём ответа бесконечно

Компонуемость и повторное использование

Один сервис - множество потребителей:



Компонуемость и повторное использование

Микросервисы - это не только внутренняя архитектурная практика, но и бизнес-модель. Многие успешные SaaS-компании по сути являются микросервисами, которые другие компании используют

Недостатки. Распределённая сложность

- Сеть надёжна
- Задержка равна нулю
- Пропускная способность бесконечна
- Сеть безопасна
- Топология не меняется
- Есть один администратор
- Стоимость передачи равна нулю
- Сеть однородна

Недостатки. Распределённая сложность. SAGA вместо ACID

- В монолите можно обернуть несколько операций в одну транзакцию базы данных. В микросервисах каждый сервис имеет свою БД - традиционные распределённые транзакции (2PC — Two-Phase Commit) слишком медленны и не масштабируются.
- Вместо этого используется паттерн SAGA - последовательность локальных транзакций с компенсирующими действиями в случае сбоя. К примеру, бронирование авиабилета + отеля + машины. Если бронирование машины не удалось - нужны компенсирующие действия - отменить бронирование отеля и авиабилета. Но отпуск уже никто не вернет ☺

Недостатки. Распределённая сложность. CAP-теорема

- В распределённой системе невозможно одновременно гарантировать Consistency (все узлы видят одинаковые данные), Availability (каждый запрос получает ответ) и Partition Tolerance (система работает при разрыве сети между узлами)
- Нужно выбрать два из трёх. На практике P неизбежно (сеть рвётся), поэтому выбор между CP и AP

Недостатки. Операционная сложность

Аспект	Монолит	Микросервисы (50 сервисов)
CI/CD пайплайны	1	50+
Конфигурации	1 файл	50+ конфигураций
Мониторинг	1 дашборд	50+ дашбордов + межсервисный
Логи	1 файл	50 потоков → агрегация
Сертификаты TLS	1	50+ (mTLS между сервисами)
On-call ротация	1 команда	Несколько команд

Недостатки. Обязательная инфраструктура для микросервисов

- Оркестрация контейнеров (Kubernetes)
- Service mesh (Istio/Linkerd) или библиотеки устойчивости
- Централизированное логирование (ELK/Loki)
- Распределенный трейсинг (Jaeger/Tempo)
- Метрики (Prometheus/Grafana)
- Управление секретами (Vault/Sealed Secrets)
- Обнаружение сервисов (Consul, k8s DNS)

Недостатки. Обязательная инфраструктура для микросервисов

- Вместо того чтобы каждая команда самостоятельно настраивала Kubernetes, мониторинг и CI/CD, создаётся внутренняя платформа, которая предоставляет стандартизированный способ создания, развёртывания и мониторинга сервисов
- Ключевой момент такой инфры и платформы – service mesh.
  - Service mesh - инфраструктурный слой, который берёт на себя сетевую коммуникацию между сервисами - mTLS (взаимная аутентификация), retry, circuit breaking, observability. Вместо того чтобы каждый сервис реализовывал эту логику - это делает sidecar-прокси (к примеру, Envoy), работающий рядом с каждым сервисом. Istio и Linkerd - основные реализации mesh

Недостатки. Тестирование и отладка

- E2E тесты - дорогие, хрупкие, медленные
- Contract тесты - проверяют совместимость API
- Интеграционные - сервис + его БД
- Юнит-тесты - быстрые, стабильные
- Новые виды тестов:
  - Contract testing (Pact) - потребитель определяет ожидания от API провайдера, а провайдер проверяет, что он им соответствует
  - Chaos testing - намеренное внесение сбоев в продакшен
  - Canary deployment - новая версия сначала получает 1-5% трафика и при проблемах автоматический откат

Недостатки. Тестирование и отладка

- Отладка
  - Один запрос → 15 сервисов → где проблема?
  - Распределенный трейсинг (trace ID проходит через все сервисы)
  - Централизированное логирование (все логи агрегируются в одном месте, связаны по trace ID)

Недостатки. Расширенная поверхность атаки

- Монолит
  - Единая точка входа, внутренние вызовы в памяти - нечего перехватывать (почти)
- Микросервисы
  - Десятки сервисов общаются по сети - каждый вызов потенциально перехватываем

Недостатки. Расширенная поверхность атаки

- Стратегия zero trust:
  - Не доверяем ничему по умолчанию, даже внутри корпоративной сети. Каждый запрос должен быть аутентифицирован и авторизован, независимо от того, откуда он приходит. В мире микросервисов это означает: даже если сервис A и сервис B работают в одном Kubernetes кластере - они всё равно должны аутентифицировать друг друга через mTLS
  - mTLS (mutual TLS) - каждый сервис аутентифицирует каждого вызывающего (не только сервер клиенту, но и клиент серверу)
  - Network Policies - Kubernetes позволяет явно указать, какой сервис может общаться с каким
  - API Gateway - единая точка входа с аутентификацией, авторизацией, rate limiting
  - Secret Management - ключи, токены, пароли хранятся в Vault, а не в коде

Когда использовать микросервисы?

- Используйте, когда:
  - Система достаточно велика, чтобы одна команда не справлялась
  - Разные части системы имеют разные требования к масштабированию
  - Нужна высокая скорость релизов (несколько деплоев в день)
  - Есть зрелая инфраструктура DevOps
  - Команды готовы к полной ответственности за свои сервисы
- Не используйте, когда:
  - Стартап на ранней стадии (домен ещё не понятен)
  - Команда < 10 человек
  - Нет CI/CD, контейнеров, мониторинга
  - «Потому что все так делают» (cargo cult)

Когда использовать микросервисы?

Если вы не можете построить хорошо структурированный монолит, что заставляет вас думать, что микросервисы - это ответ?

Саймон Браун

- Рекомендация Мартина Фаулера - «MonolithFirst»
  - Начните с монолита, выделяйте микросервисы по мере роста и появления реальных проблем. На ранней стадии вы не знаете, где пройдут границы сервисов. Неправильно проведённые границы - хуже, чем монолит, потому что перемещение функциональности между микросервисами значительно сложнее, чем рефакторинг модулей в монолите

Сервисы VS Микросервисы

Параметр	SOA	Микросервисы
Межсервисное взаимодействие	Умные каналы, такие как сервисная шина предприятия, с использованием тяжеловесных протоколов вроде SOAP и других веб-сервисных стандартов	Примитивные каналы, такие как брокер сообщений, или прямое взаимодействие между сервисами с помощью легковесных протоколов наподобие REST или gRPC
Данные	Глобальная модель данных и общие БД	Отдельные модель данных и БД для каждого сервиса
Типовой сервис	Крупное монолитное приложение	Небольшой сервис

Будут вопросы? Не стесняйтесь!